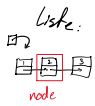
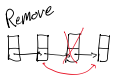
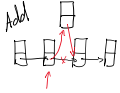


Hacked datastructure (linked)

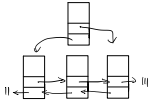


@dataclass
class linked_list:
 first: node
 last: int

@dataclass
class node:
 val: int
 next: #node

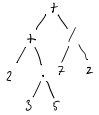


double linked list



Abstract syntax tree

$(2 + (3 \cdot 5)) + (7/2)$



$t(+(2 \cdot (3 \cdot 5)) (/ 7 2))$ val, (left), (right)
 $(2 + (3 \cdot 5)) + (7/2)$ (left) val, (right)
 $(2(3 \cdot 5) +) (7/2) +$ (left), (right), val
 $2 \cdot 3 \cdot 5 + 7/2 +$

Polish notation

def make_list() -> linked_list
 """ makes an empty list """
 return linked_list(None, 0)

def remove(w: linked_list, i: int) -> None:
 """ Removes the element at place i.
 on the list w. Requires 0 <= i < len(w). """

if i == 0:
 w.first = w.first.next

else:
 _remove(w.first, i-1)

w.len = w.len - 1

def _remove(u: node, i: int) -> None:

if i == 0:
 u.next = u.next.next

else:
 _remove(u.next, i-1)

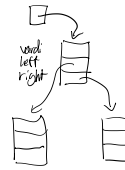
def add(v: int, w: linked_list, u: int) -> None:
 """ Adds v to w at place u, 0 <= u < len(w). """
 if u == 0:
 w.first = Node(v, w.first)
 else:
 _add(v, w.first, u-1)
 w.len = w.len + 1

def _add(v: int, n: node, i: int) -> None:

if i == 0:
 n.next = Node(v, n.next)

else:
 _add(v, n.next, i-1)

Binære træer



@dataclass
class tree:
 root: #Node

@dataclass
class node:
 val: int
 left: #Node
 right: #Node

def tree_to_list(t: tree) -> list[int]:
 """ ... """

if root = None:
 return []

else:
 return _tree_to_list(t.root)

def _tree_to_list(n: node) -> list[int]:

if n = None:
 return []

else:
 return [n.val] + _tree_to_list(n.left) + _tree_to_list(n.right)